Engineering Applications of Artificial Intelligence 98 (2021) 104116

Contents lists available at ScienceDirect

Engineering Applications of Artificial Intelligence

journal homepage: www.elsevier.com/locate/engappai

Learning dynamic regression with automatic distractor repression for real-time UAV tracking

Changhong Fu [a,*], Fangqiang Ding [a], Yiming Li [a], Jin Jin [a], Chen Feng [b]

[a] School of Mechanical Engineering, Tongji University, Shanghai 201804, China
[b] Tandon School of Engineering, New York University, New York NY 10012, United States

ARTICLE INFO

Keywords:
Unmanned aerial vehicle
Visual object tracking
Discriminative correlation filter
Learning dynamic regression
Local maximums repression

ABSTRACT

With high efficiency and efficacy, the trackers based on the discriminative correlation filter have experienced rapid development in the field of unmanned aerial vehicle (UAV) over the past decade. In literature, these trackers aim at solving a regression problem in which the circulated samples are mapped into a Gaussian label for online filter training. However, the fixed target label for regression makes trackers lose adaptivity in uncertain tracking scenarios. One of the typical failure cases is that the distractors, *e.g.*, background clutter, camouflage, and similar object, are prone to confuse these trackers. In this work, an efficient approach to instantly monitor the local maximums of the response map for discovering distractors automatically is proposed. In addition, the regression target is accordingly learned, *i.e.*, the location possessing local maximum indicates latent distractor and thus should be repressed by reducing its target response value in filter training. Qualitative and quantitative experiments performed on three challenging well-known benchmarks demonstrate that the presented method not only outperforms the state-of-the-art handcrafted feature-based trackers but also exhibits comparable performance compared to deep learning-based approaches. Specifically, the presented tracker has phenomenal practicability in real-time UAV applications with an average speed of ~50 frames per second on an affordable CPU.

1. Introduction

Visual object tracking is among the most vital research topics in machine vision with wide-range applications, including video surveillance (Pantrigo et al., 2010), expression recognition (Khan et al., 2017), and robot cooperative localization (Ahmad et al., 2017), *etc.* Given the object state denoted by a bounding box in the first image, the objective of generic object tracking is to estimate the object state in the following frames. Due to the scarcity of online training samples and unanticipated object appearance variations like deformation, rotation, and viewpoint change, developing a robust and accurate visual tracking algorithm is still a non-trivial task.

Recently, with the flourishment of unmanned aerial vehicle (UAV), providing UAV with visual tracking function has yielded numerous applications in robotics, *e.g.*, aerial cinematography (Karakostas et al., 2019), pose estimation (Li et al., 2020a), and obstacle avoidance (Bharati et al., 2018). Apart from generic object tracking in which images are usually captured from stationary cameras, UAV object tracking consumes images acquired by the aerial mobile camera from a higher shoot altitude. Consequently, more challenging scenarios including fast object/UAV motion, distractions in the background with turbulent change, blur, low resolution, *etc.*, have emerged in UAV visual tracking community.

In literature, deep learning has been utilized to improve tracking performance. Two mainstreams are: (a) construct more discriminative object appearance models using deep features extracted from convolutional neural network (CNN) (Danelljan et al., 2016; Dai et al., 2019; Zhao et al., 2019; Li et al., 2020b), (b) design end-to-end deep architectures suitable for tracking tasks (Bertinetto et al., 2016; Li et al., 2019; Kuai et al., 2019; Gao et al., 2020). However, complex convolution operations encumber the execution efficiency of deep trackers and discount their practicability on the mobile platform with limited computation resources like UAV.

Another research interest of the visual tracking community is employing cheap handcrafted features to build the appearance model in the discriminative correlation filter (DCF) (Li et al., 2020a; Bertinetto et al., 2016; Henriques et al., 2015; Wang et al., 2018; Huang et al., 2019; Fu et al., 2020). Basically, DCF intends to learn a foreground–background classifier online by solving a regression problem in which the circulated samples are regressed into a Gaussian label. The highly efficient dot-product operation in the frequency domain is used as an alternative method of complex cyclic correlation in the spatial domain, so DCF-based trackers achieve high processing speed only on a low-cost CPU which are ideal choices for UAV tracking applications. However,

* Corresponding author.
E-mail address: changhongfu@tongji.edu.cn (C. Fu).

https://doi.org/10.1016/j.engappai.2020.104116
Received 16 May 2020; Received in revised form 5 November 2020; Accepted 26 November 2020
Available online xxxx
0952-1976/© 2020 Elsevier Ltd. All rights reserved.

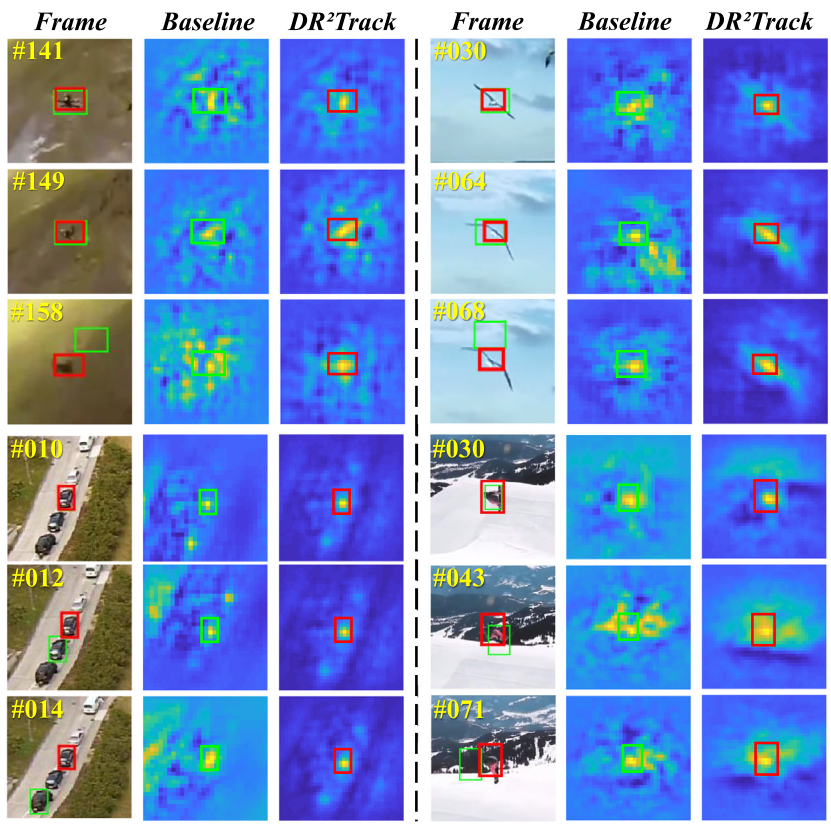

Fig. 1. Visualized estimated results and response maps of the baseline tracker (Li et al., 2018) and DR^{2}Track in *ChasingDrones, Gull2, Car2*, and *SnowBoarding6* sequence (from top left to bottom right). In the original images, **Red** bounding boxes denote the output of DR^{2}Track, while **Green** ones are the estimated results of the baseline. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

due to the deficiency of handcrafted features, there exists a clear gap between the tracking performance of current handcrafted feature-based trackers with that of deep trackers. Therefore, it is worthwhile thinking about: *can we offset the gap with deep trackers or even transcend them only using DCF framework with handcrafted features?* If we can, such kind of methods can remarkably advance the real-world UAV tracking applications in industry.

To answer the above question, let us firstly review the pipeline of classical DCF methods (Henriques et al., 2015; Bolme et al., 2010; Li and Zhu, 2014) which can be disassembled into two core components: object detection and filter training. For an upcoming image, the current filter is utilized to execute the classification task on the search region and to output a response map in which the score of each pixel denotes the localization possibility of the tracked object. Thus the object location can be estimated by finding the peak value of the response. Based on the estimated location, the training sample is extracted and applied in regression learning to train a correlation filter as a foreground-background classifier. The regression objective is to minimize the disparity of a Gaussian label and the correlation between the filter and training sample.

In general, the tracking accuracy depends heavily on the quality of response maps. For example, when the response value distribution within the object area is nearly a Gaussian shape and all the values in the background are close to zero, the object location can be precisely estimated. However, in practice, there are generally few aforementioned phenomena, especially in the scenarios with plenty of distractors,[1] *e.g.*, background clutter, camouflage, and similar object around. In such challenging scenarios, the response value of distractors can be obviously larger than that of surrounding pixels, thereby producing local maximums which are very possible to increase gradually and lead to detrimental loss of the target, as demonstrated in Fig. 1.

To this end, we propose a novel tracker learned by *Dynamic Regression with Automatic Distractor Repression* (DR^{2}Track). The regression label is adjusted dynamically to repress distractors indicated as the local maximums on the fly. In this way, distractors can be specially taken care of in the filter training. As a consequence, the object can be discriminatingly localized in the clutter background because the shape of response maps is more concentrated and less dispersive, as illustrated in Fig. 1. Moreover, the search scope in object detection can be expanded without worrying about bringing more distractions. To our excitement, extensive and exhaustive experimental results show the superiority of DR^{2}Track against the state of the arts and that it achieves an over real-time tracking speed on an affordable CPU. Without whistles and bells, DR^{2}Track can meet the requirements in both tracking accuracy and speed for UAV tracking. Our contributions are three-fold:

- A novel adaptive and automatic distractors discovery algorithm is developed. Local maximum values of the response map are exploited as the latent distractors during tracking.
- A new dynamic regression problem is established for filter training to advance tracking performance. The regression target is dynamically changed to repress the response of distractors, thereby improving robustness significantly.
- Exhaustive and extensive experiments conducted on three well-known challenging UAV benchmarks have validated the practicability and superiority of DR^{2}Track under the real-world UAV tracking scenarios compared to state-of-the-art tracking algorithms, as shown in Fig. 2.

[1] For conciseness, we define the pixel in the background which generates high response score with the correlation filter as *distractor*.

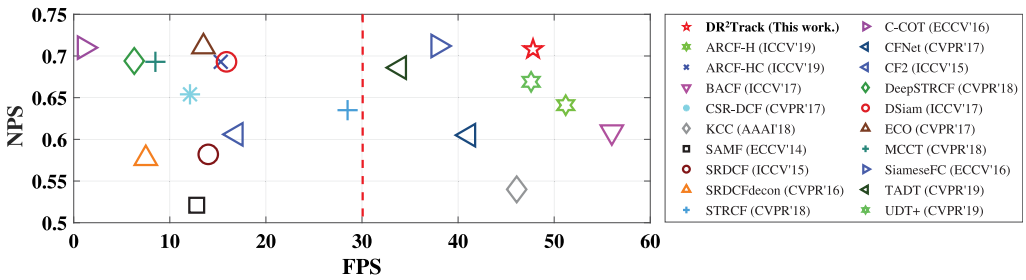

Fig. 2. The ranking plot of twenty-two evaluated trackers on average FPS and NPS (a metric in this paper) in three benchmarks. Handcrafted feature-based trackers (left column in the legend) are implemented only with CPU while CNN-based trackers (right column in the legend) are operated with GPU acceleration. With an average speed of ~50 fps on a low-cost CPU, DR^{2}Track outperforms all evaluated handcrafted feature-based trackers, and also shows comparable performance compared to CNN-based methods. **Red** line (at 30FPS) denotes the threshold of real time which is a significant criterion to evaluate the practicability of trackers in UAV tracking scenarios. Note that real-world UAV tracking scenarios are more challenging than generic ones. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

2. Related work

2.1. Visual tracking methods

In literature, visual object tracking methods generally pertain to either generative methods or discriminative methods. Generative methods tend to learn a generative model and localize the object by minimizing the reconstruction error. Stepping into this century, extensive researches have been carried out to boost the performance of generative approaches in various branches, *e.g.*, mean shift (Comaniciu et al., 2000), particle filter (Li et al., 2010), sparse and subspace representations (Mei and Ling, 2009; Ross et al., 2004). Despite exploiting prior probability, generative methods are developed based on the assumption that the object appearance seldom changes and merely extract object feature information, thereby failing to tackle complicated tracking scenarios, and hence are gradually replaced by discriminative methods in recent years.

Under the supervision of classification labels, the discriminative model can be easily constructed by learning to classify the negative and positive samples. Owing to their flexible decision boundary, discriminative approaches have high robustness and accuracy against various difficulties. Babenko et al. (2009) and Grabner et al. (2008) respectively employed multiple instance learning and multi-classifier combination to effectively alleviate noise and improve the tracking robustness. To avoid generating inaccurate samples, Hare et al. (2011) viewed the overall tracking task as a structured output prediction. With the extensive success achieved by deep learning, many state-of-the-art trackers (Held et al., 2016; Valmadre et al., 2017; Guo et al., 2017) applied large-scale datasets, including images and classification labels to train end-to-end tracking system off-line. Though having gained appealing results, such trackers run at a high-end GPU for acceleration and fail to run in real-time on a low-cost CPU. Therefore, they cannot be applied on platforms, UAV in particular, with limited payload, power consumption as well as computation capability.

As the seminal work, Bolme et al. (2010) firstly introduced the DCF framework into the visual tracking community. With both fair robustness and light calculation, DCF-based trackers (Danelljan et al., 2017a, 2015; Henriques et al., 2015; Dai et al., 2019; Bertinetto et al., 2016) have attracted extensive attention from researchers. In the following subsection, the development of DCF-based methods will be reviewed.

2.2. DCF-based trackers

The greatest strength of DCF-based methods is their spectacular running speed, for example, the tracker proposed in an early work (Bolme et al., 2010) can process ~1000 frames per second (FPS) only on a cheap CPU. However, this forerunner cannot satisfy the needs of high-precision and strong-robustness tracking.

In recent years, numerous works have attempted to elevate the tracking performance of DCF-based trackers. J. F. Henriques et al. rephrased the original tracker in Bolme et al. (2010) as an instance of ridge regression with linear kernel in Henriques et al. (2015), where they also introduced one with the non-linear kernel. Mapping the samples to the high-dimension space, the non-linear kernel can greatly advance the distinguishing ability of the filter. In terms of the feature representations with multi-channel handcrafted features, histograms of gradient (HOG) (Dalal and Triggs, 2005) and color names (CN) (Weijer and Schmid, 2006) are introduced into correlation filters in Henriques et al. (2015) and Danelljan et al. (2014), respectively. To exploit the semantic information in images, Ma et al. (2015) interpreted the hierarchical convolutional features to appearance modeling, while Qi et al. (2016) boosted the tracking performance by combining multiple convolutional filters.

In order to estimate the scale variations, Li and Zhu (2014) and Danelljan et al. (2004) gave two solutions which were followed by most DCF-based methods after that. In addition, Danelljan et al. (2017b) proposed two strategies, *i.e.*, dimensionality reduction and sub-grid interpolation, to reduce the calculation cost for scale adaptivity. Having achieved state-of-the-art performance, Danelljan et al. (2017a) utilized many elaborate techniques, *e.g.*, factorized convolutional operator, generative sample model, and sparse update policy, based on its prior work Danelljan et al. (2016) that firstly introduced multi-resolution sampling. However, the aforementioned trackers all adopted fixed Gaussian label in the regression solving and cannot handle distractors adaptively and automatically.

2.3. Distractor-aware methods in object tracking

In visual object tracking, one of the main reasons for tracking failure is the interference of distractors. If not repressed in advance, these distractors will distract the attention of the classifier and the estimated locations will drift to these distractors. Mueller et al. (2017) proposed to concatenate a novel context term to the rigid regression equation to further restrict the response of context information to zero. Meantime, in Galoogahi et al. (2017), the large patch was sheared into little patches with the same size as the object, in this way, the suppression of distractors was enhanced. Though background distraction can be repressed to some extent, the aforementioned works cannot precisely locate the distractor in an automatic manner, and the effective as well as adaptive distractor repression techniques remain to be investigated.

When the distractors emerge in the search region or get closer to the object, drastic variations usually happened to the filter coefficients which severely hamper its classification steadily. To cope with these abnormal variations, Li et al. (2018) investigated the temporal constraints of the filters in the training phase, while Huang et al. (2019) presented to restrict the aberrance emergence by adding a response-map-consistency term in the objective function. However, the repression of global changes in the above methods cannot address the local distraction. Furthermore, the constant restriction in correlation filter or response variation is lack of generalization capability, that is,

an overly low restriction strength is not effectual and an overly high one can easily lead to over-fitting.

In Xiao et al. (2016), distractors were detected via multi-level clustering and a global constrain was utilized to dynamically handle them. Kuai et al. (2018) combined Siamese networks with kernelized correlation filters with distractors detection as an intermediate phase. Nevertheless, the performance of Xiao et al. (2016) and Kuai et al. (2018) cannot meet the requirements of precise and real-time tracking on CPU. In this work, the pixel-level distractors can be timely suppressed on the fly, and powerful filters can be constructed with real-time speed on an affordable CPU.

2.4. UAV-oriented object tracking methods

In the past decade, UAV commerce has seen rapid development and increasing laboratories have paid a lot to research intelligent, affordable, and flexible drones. As a core field in computer vision, visual object tracking can make drones capable of high-level applications. Therefore, it is imperative to devise UAV-oriented object tracking methods and design evaluation benchmarks to meet incremental needs. However, only a few works have focused on this area. Mueller et al. (2016), Du et al. (2018) and Li and Yeung (2017) respectively proposed a benchmark to evaluate object tracking methods on UAV platform. To address partial occlusion, Fu et al. (2018) utilized part-based filter sets for multiple response map fusion to alleviate model degradation. In order to take advantage of history information, Li et al. (2020b) proposed a key-frame technique to compensate for exponential decay of memory. Fu et al. (2019) adopted consensus scoring to avoid learning drastic variations. Despite appealing tracking performance in UAV tracking datasets, these trackers all employ CNN features for appearance representations, and cannot run in real-time even on GPU which conflicts with the weak power capability and limited computation resources of UAV. Only using handcrafted features and effectively repressing distractors in complex scenarios, the presented method has superior in both speed and precision and hence is a better choice for UAV tracking.

3. Preliminary

3.1. Discriminative correlation filter (DCF)

Typical DCF-based tracker (Bertinetto et al., 2016; Henriques et al., 2015; Danelljan et al., 2017b) tried to learn a discriminative filter by restricting the result of cyclic convolution between the filter and sample to a presetting Gaussian label. As is known, the cyclic convolution in the spatial domain can be transferred to the element-wise product in the frequency domain. Given this, the regression equation of the DCF tracker can be solved in the frequency domain rapidly while maintaining abundant synthetic training samples. As for the tth frame, the filter $\mathbf{f}_t = [\mathbf{f}_t^1, \mathbf{f}_t^2, \mathbf{f}_t^3, \dots, \mathbf{f}_t^K] \in \mathbb{R}^{L\times K}$ can be obtained by:

$$\arg\min_{\mathbf{f}_t} \frac{1}{2}\Big\| \sum_{k=1}^{K} \mathbf{f}_t^k \circledast \mathbf{s}_t^k - \mathbf{y}\Big\|_2^2 + \frac{\alpha}{2}\sum_{k=1}^{K} \|\mathbf{f}_t^k\|_2^2 , \tag{1}$$

where $\mathbf{s}_t = [\mathbf{s}_t^1, \mathbf{s}_t^2, \mathbf{s}_t^3, \dots, \mathbf{s}_t^K] \in \mathbb{R}^{L\times K}$ denotes the extracted feature representations across K feature channels of the cropped patch which centers at the predicted location in current frame and $\mathbf{y} \in \mathbb{R}^L$ is the Gaussian regression label which can be written by $\mathbf{y}(l) = exp(\frac{-(l-L)^2}{2\sigma^2})$ $(l = 1, 2, 3, \dots, L)$, where σ denotes a fixed variance factor. L is the length of vectorized feature representations in each channel and $\circledast$ symbolizes the cyclic convolution operator. The term at the back has the function of avoiding over-fitting, in which α is a hyper parameter. Traditionally, a linear interpolation strategy is adopted to update the model which can be expressed by:

$$\widehat{\mathbf{F}}_t = (1-\theta)\widehat{\mathbf{F}}_{t-1} + \theta\widehat{\mathbf{f}}_t , \tag{2}$$

where $\hat{\cdot}$ symbolizes the discrete Fourier transform (DFT) of a signal, $\widehat{\mathbf{F}}_t \in \mathbb{C}^{L\times K}$ denotes the model used for detection in frame t+1 and $\widehat{\mathbf{f}}_t$ is the solution of Eq. (1) in the frequency domain. During the detection stage, the updated model is utilized to search for the optimal object location in the upcoming image via generating a response map $\mathbf{M}_{t+1}$ by:

$$\mathbf{M}_{t+1} = \mathcal{F}^{-1}\Big(\sum_{k=1}^{K} \widehat{\mathbf{F}}_t^k \odot \widehat{\mathbf{z}}_t^k\Big) , \tag{3}$$

where $\mathbf{z}_t^k$ denotes the feature representations in the kth channel of search region, $\odot$ denotes the element-wise product operation, and $\mathcal{F}^{-1}$ represents the inverse discrete Fourier transform (IDFT). The new object location can be obtained by seeking out the peak location of the response map.

3.2. Spatially regularized discriminative correlation filter (SRDCF)

In Henriques et al. (2015), the search area is only 2.5 times larger than the object area which restricts the filter discrimination improvement due to inevitable over-fitting. By exploiting an enlarged search region (16 times bigger than the object), the SRDCF tracker (Danelljan et al., 2015) owns a more balanced sample distribution. However, expanding the search scope aggravates the boundary effect for bringing more inaccurate negative samples, which makes the classification scores only accurate in the center area of the patch. In view of this, the spatial regularization is presented to be added into the training objective which can be written by:

$$\arg\min_{\mathbf{f}_t} \frac{1}{2}\Big\| \sum_{k=1}^{K} \mathbf{f}_t^k \circledast \mathbf{s}_t^k - \mathbf{y}\Big\|_2^2 + \frac{\alpha}{2}\sum_{k=1}^{K} \|\mathbf{w} \odot \mathbf{f}_t^k\|_2^2 , \tag{4}$$

where $\mathbf{w} \in \mathbb{R}^L$ symbolizes the spatial regularization vector used to penalize the coefficients magnitude in the filter, especially in the background area. By introducing the spatial penalty, the influence of the background region can be weakened. In other words, the boundary effect is alleviated in the detection stage while expanding the search area apparently.

3.3. Spatial–temporal regularized correlation filter (STRCF)

Different to linear interpolation model update in Henriques et al. (2015) and Danelljan et al. (2015), Li et al. (2018) proposed a temporal regularization to adaptively update the filter in the training phase. By adding the minimization subtask of filter successiveness, the training objective can be expressed as follows:

$$\arg\min_{\mathbf{f}_t} \frac{1}{2}\Big\| \sum_{k=1}^{K} \mathbf{f}_t^k \circledast \mathbf{s}_t^k - \mathbf{y}\Big\|_2^2 + \frac{\alpha}{2}\sum_{k=1}^{K} \|\mathbf{w} \odot \mathbf{f}_t^k\|_2^2 + \frac{\eta}{2}\sum_{k=1}^{K} \|\mathbf{f}_t^k - \mathbf{f}_{t-1}^k\|_2^2 , \tag{5}$$

where η denotes the fixed temporal regularization parameter and $\mathbf{f}_{t-1}^k$ is the filter in the last frame. This kind of update strategy can avoid the severe degradation of the filter, for restricting the filter coefficients variations effectively. Due to the appealing performance of the STRCF tracker, we choose it as the baseline tracker and develop our tracker as follows.

4. Existing problems

4.1. Drawback of fixed regression target

Most DCF-based methods utilize a fixed regression target that is always a Gaussian function, as shown in Fig. 3. Although robust to most normal tracking scenarios, filters trained in this way cannot cope with distractors existing in the surrounding area. According to the implicit cyclic shift of the original sample, each element of the target label denotes the desired value of the matrix multiplication between the synthetic training samples obtained by corresponding shift length

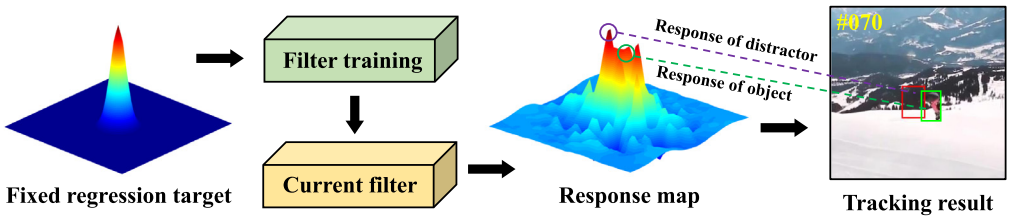

Fig. 3. An instance of the tracking output of the baseline tracker (Li et al., 2018) which utilizes fixed Gaussian-shaped regression target for filter tracking. **Red** bounding box denotes the output of the baseline tracker, while **Green** one is the groundtruth output. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

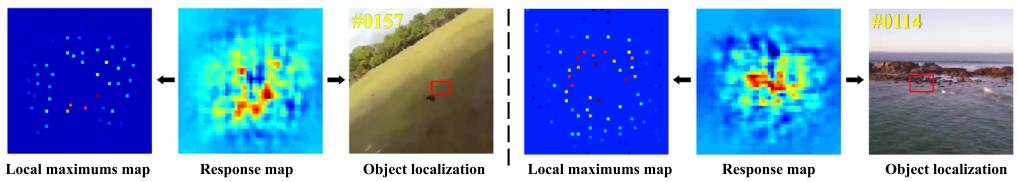

Fig. 4. Examples of model drift caused by the presence of local maximums in the response map which generated in the tracking process of the baseline tracker (Li et al., 2018). **Red** bounding boxes are the output results. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

and the filter. Similarly, the response map exhibits one-to-one response results of the circulated search samples with the current filter, as shown in Fig. 5. Their mathematics relationship can be expressed by $\mathbf{y}(l) \rightarrow \sum_{k=1}^{K} \mathbf{s}^{k\top}[\tau_l]\mathbf{f}^{k*}$ and $\mathbf{M}(l) = \sum_{k=1}^{K} \mathbf{z}^{k\top}[\tau_l]\mathbf{f}^{k*}$. Here, $\cdot^*$ and $[\tau_l]$ symbolizes the complex conjugate and shift operator. Generally, the desired values of the training samples centering at the pixels in the background area are relatively low. However, the real response values in the locations of distractors usually exceed the desired values greatly and even transcend the response values in object area due to the similar appearance between distractors and the object, which can be seen in the response map of the baseline tracker (Li et al., 2018) in Fig. 3. Therefore, the fixed regression target cannot meet the requirements to distinguish the distractors and object by conventional filter training. In other words, training filter with fixed regression label fails to tackle distractors, thus resulting in model drift or even tracking failure easily.

4.2. Local maximum values in response map

In the ideal conditions, the values in the background region of the response map should be very small, for the filter classifies the samples centering at the background region as negative samples. Yet, the real distribution of the response map is disordered and there are numbers of local maximums. In most cases, the values of these local maximums are lower than the global maximum value, but they usually indicate the presence of distractors that have a similar appearance to the object. Sometimes they can surpass the peak as the tracking process goes on and lead to model drift, as exhibited in Fig. 4. Concluding from the aforementioned analysis, these local maximums are abnormal stuff in the tracking process and are serious threats for stable object tracking, and hence should be carefully and precisely repressed.

5. Proposed methods

5.1. Training objective of proposed DR^{2}Track

In order to offset the defects of a fixed regression target, we propose to adaptively adjust the target label frame-by-frame depending on the response map generated in the detection stage. The distractors are suppressed by monitoring the local maximums in the response map and dynamically penalizing the corresponding values in the target label. Before the training phase, a distractor-suppressed vector $\mathbf{m}_t \in \mathbb{R}^L$ is calculated using the response map obtained in the detection phase by:

$$\mathbf{m}_t(l) = 1 - \frac{\beta}{\phi}\Psi\left(\mathbf{M}[\tau_i]\right)(l)\ , \quad 1 + \frac{C}{2} \le l \le L - \frac{C}{2}\ . \tag{6}$$

Here, β symbolizes a hyper parameter to govern the suppression degree, ϕ denotes the maximum value of the response map and $[\tau_i]$ is used for response map shift to match the peak location with the target label. C denotes the length of the object area ($C \ll L$ in our method) and $\Psi(\cdot)$ is a local-maximum-extracted function which crops out all local maximums and sets other elements to zero, as shown in Fig. 5. Following the spatial–temporal regularization in the baseline tracker (Li et al., 2018), our novel training objective can be expressed as follows:

$$\arg\min_{\mathbf{f}_t} \left\| \sum_{k=1}^{K} \mathbf{f}_t^k \circledast \mathbf{s}_t^k - \mathbf{m}_t \odot \mathbf{y} \right\|_2^2 + \sum_{k=1}^{K} \left(\|\mathbf{w} \odot \mathbf{f}_t^k\|_2^2 + \eta \|\mathbf{f}_t^k - \mathbf{f}_{t-1}^k\|_2^2 \right)\ . \tag{7}$$

Remark 1. Considering that the objective of our method is to repress the distractors in the background area, we are not interested in the local maximums in the object area. In addition, we only take the top E local maximums in the background area into account in this work. Specifically, in the first frame, we set all elements in $\mathbf{m}_1$ as 1 with regard to the uncertainty of local maximum locations.

5.2. Iterative optimization for filter learning

In consideration of the inherent bi-convex characteristic of Eq. (7), the alternative direction method of multipliers (ADMM) algorithm can be used to iteratively obtain the optimum solution in global. By introducing a auxiliary variable $\mathbf{h}_t \in \mathbb{R}^{L\times K}$ and making it subject to $\mathbf{h}_t = \mathbf{f}_t$. With two variables, *i.e.*, $\mathbf{f}_t$ and $\mathbf{h}_t$, the Eq. (7) can be rewritten by:

$$\begin{aligned} \arg\min_{\mathbf{h}_t,\mathbf{f}_t} & \frac{1}{2}\left\| \sum_{k=1}^{K} \mathbf{h}_t^k \circledast \mathbf{s}_t^k - \mathbf{m}_t \odot \mathbf{y} \right\|_2^2 + \frac{1}{2}\sum_{k=1}^{K} \|\mathbf{w} \odot \mathbf{f}_t^k\|_2^2 \\ & + \frac{\eta}{2}\sum_{k=1}^{K} \|\mathbf{h}_t^k - \mathbf{h}_{t-1}^k\|_2^2 \qquad s.t. \quad \mathbf{h}_t = \mathbf{f}_t \end{aligned}\ . \tag{8}$$

By introducing the Lagrangian multiplier $\mathbf{d}_t \in \mathbb{R}^{L\times K}$ and penalty factor ρ, the Augmented Lagrangian form of Eq. (8) can be formulated as:

$$\begin{aligned} \arg\min_{\mathbf{h}_t,\mathbf{f}_t,\mathbf{d}_t} & \frac{1}{2}\left\| \sum_{k=1}^{K} \mathbf{h}_t^k \circledast \mathbf{s}_t^k - \mathbf{m}_t \odot \mathbf{y} \right\|_2^2 + \frac{1}{2}\sum_{k=1}^{K} \|\mathbf{w} \odot \mathbf{f}_t^k\|_2^2 \\ & + \frac{\eta}{2}\sum_{k=1}^{K} \|\mathbf{h}_t^k - \mathbf{h}_{t-1}^k\|_2^2 + \sum_{k=1}^{K} \mathbf{d}_t^{k\top}(\mathbf{h}_t^k - \mathbf{f}_t^k) + \frac{\rho}{2}\sum_{k=1}^{K} \|\mathbf{h}_t^k - \mathbf{f}_t^k\|_2^2 \end{aligned}\ . \tag{9}$$

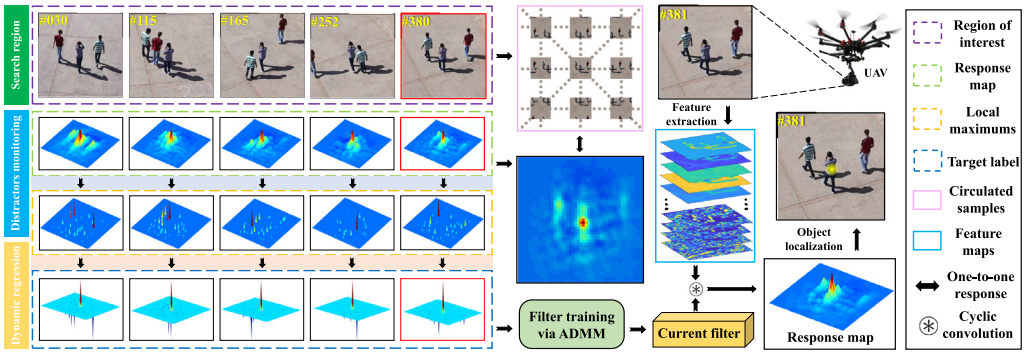

Fig. 5. The object tracking workflow of DR^{2}Track. Centering at the calculated current search center, we crop a region of interest for each upcoming image and execute handcrafted feature extraction. By cyclic convolution operation between the feature representation and the current filter, the response map is generated. Utilizing the center offset of the peak and combine the current search center, we can decide the optimum location of the current object. For latent distractors repression, the local maximum points in the background region of response map are primarily extracted and the regression label can be dynamically learned referring to it based on the fixed Gaussian label. We finally update the filter by iterative optimization of ADMM. Note that we shift the peak of the regression label from the corner to the center for the convenience of understanding.

Here, two added terms at the back of Eq. (9) improve the convexity of the algorithm, and hence elevate the optimization ability greatly. For simplicity of this equation, $\mathbf{g}_t = \frac{\mathbf{d}_t}{\rho}$ is introduced. Then the last two terms of Eq. (9) can be derived as follows:

$$\begin{aligned}
&\sum_{k=1}^{K} \mathbf{d}_t^{k\top}(\mathbf{h}_t^k - \mathbf{f}_t^k) + \frac{\rho}{2}\sum_{k=1}^{K} \|\mathbf{h}_t^k - \mathbf{f}_t^k\|_2^2 \\
=&\frac{\rho}{2}\sum_{k=1}^{K}\left(2\mathbf{g}_t^{k\top}(\mathbf{h}_t^k - \mathbf{f}_t^k) + (\mathbf{h}_t^k - \mathbf{f}_t^k)^\top(\mathbf{h}_t^k - \mathbf{f}_t^k)\right) \\
=&\frac{\rho}{2}\sum_{k=1}^{K}\left((\mathbf{h}_t^k - \mathbf{f}_t^k + 2\mathbf{g}_t^k)^\top(\mathbf{h}_t^k - \mathbf{f}_t^k)\right) \\
=&\frac{\rho}{2}\sum_{k=1}^{K}\left((\mathbf{h}_t^k - \mathbf{f}_t^k + \mathbf{g}_t^k)^\top(\mathbf{h}_t^k - \mathbf{f}_t^k) + (\mathbf{h}_t^k - \mathbf{f}_t^k)^\top\mathbf{g}_t^k\right)
\end{aligned} \tag{10}$$

For further simplicity of the equation, an extra $\mathbf{g}_t^{k\top}\mathbf{g}_t^k$ is added at the end of Eq. (10):

$$\begin{aligned}
&\frac{\rho}{2}\sum_{k=1}^{K}\left((\mathbf{h}_t^k - \mathbf{f}_t^k + \mathbf{g}_t^k)^\top(\mathbf{h}_t^k - \mathbf{f}_t^k) + (\mathbf{h}_t^k - \mathbf{f}_t^k + \mathbf{g}_t^k)^\top\mathbf{g}_t^k\right) \\
=&\frac{\rho}{2}\sum_{k=1}^{K}\left((\mathbf{h}_t^k - \mathbf{f}_t^k + \mathbf{g}_t^k)^\top(\mathbf{h}_t^k - \mathbf{f}_t^k + \mathbf{g}_t^k)\right) \\
=&\frac{\rho}{2}\sum_{k=1}^{K}\|\mathbf{h}_t^k - \mathbf{f}_t^k + \mathbf{g}_t^k\|_2^2
\end{aligned} \tag{11}$$

It is noted that this extra term has no impact on the final solution because we only obtain the partial derivative of $\mathbf{h}_t$ and $\mathbf{f}_t$ other than $\mathbf{g}_t^k$ during the overall solving process. Given the derivation result from the above equation, Eq. (9) can be reformulated as:

$$\begin{aligned}
\arg\min_{\mathbf{h}_t,\mathbf{f}_t,\mathbf{g}_t} &\frac{1}{2}\left\|\sum_{k=1}^{K}\mathbf{h}_t^k \circledast \mathbf{s}_t^k - \mathbf{m}_t \odot \mathbf{y}\right\|_2^2 + \frac{1}{2}\sum_{k=1}^{K}\|\mathbf{w}\odot\mathbf{f}_t^k\|_2^2 \\
&+\frac{\eta}{2}\sum_{k=1}^{K}\|\mathbf{h}_t^k - \mathbf{h}_{t-1}^k\|_2^2 + \frac{\rho}{2}\sum_{k=1}^{K}\|\mathbf{h}_t^k - \mathbf{f}_t^k + \mathbf{g}_t^k\|_2^2
\end{aligned} \tag{12}$$

The core idea of the ADMM algorithm is to decompose the complex main problem into multiple simple subproblems and iteratively solve them with other variables fixed. In our condition, we can respectively obtain optimum $\mathbf{h}_t, \mathbf{f}_t$ and $\mathbf{g}_t$ by iteratively solving three subproblems for U times which are detailed as follows.

5.2.1. Solving subproblem $\mathbf{h}_t$

For a new iteration in frame t, we primarily optimize the subproblem $\mathbf{h}_t$ with $\mathbf{f}_t$ and $\mathbf{g}_t$ fixed. Having obtained $\mathbf{g}_{t,u-1}$ and $\mathbf{f}_{t,u-1}$ in the last iteration, the newly solved $\mathbf{h}_{t,u}$ of current subproblem in the uth $(u = 1, 2, \ldots, U)$ iteration can be gained by:

$$\begin{aligned}
\arg\min_{\mathbf{h}_t} &\frac{1}{2}\left\|\sum_{k=1}^{K}\mathbf{h}_t^k \circledast \mathbf{s}_t^k - \mathbf{m}_t \odot \mathbf{y}\right\|_2^2 + \frac{\eta}{2}\sum_{k=1}^{K}\|\mathbf{h}_t^k - \mathbf{h}_{t-1}^k\|_2^2 \\
&+\frac{\rho_u}{2}\sum_{k=1}^{K}\|\mathbf{h}_t^k - \mathbf{f}_{t,u-1}^k + \mathbf{g}_{t,u-1}^k\|_2^2
\end{aligned} \tag{13}$$

Here, in consideration that the variables in the second block of Eq. (12) are all assumed to be constants in this subproblem, we only retain the other three blocks for optimization. Specificity, we set $\mathbf{g}_{t,u-1}$ and $\mathbf{f}_{t,u-1}$ as zero vector for the first iteration. Also, the temporal regularization parameter η is set to zero in the first frame of the current image stream.

Solving Eq. (13) directly by letting its derivative to zero is computationally complex due to cyclic convolution operation and multiple channels composition. Following traditional tricks in DCF-based methods, we transfer the overall equation to the frequency domain and calculate it in a more efficient way. The frequency form of Eq. (13) is reformulated as:

$$\begin{aligned}
\arg\min_{\hat{\mathbf{h}}_t} &\frac{1}{2}\left\|\sum_{k=1}^{K}\hat{\mathbf{h}}_t^k \odot \hat{\mathbf{s}}_t^k - \widehat{\mathbf{m}_t \odot \mathbf{y}}\right\|_2^2 + \frac{\eta}{2}\sum_{k=1}^{K}\|\hat{\mathbf{h}}_t^k - \hat{\mathbf{h}}_{t-1}^k\|_2^2 \\
&+\frac{\rho_u}{2}\sum_{k=1}^{K}\|\hat{\mathbf{h}}_t^k - \hat{\mathbf{f}}_{t,u-1}^k + \hat{\mathbf{g}}_{t,u-1}^k\|_2^2
\end{aligned} \tag{14}$$

Due to the feature channel complexity of the first block in Eq. (14), solving the value of every pixel in every channel is time-consuming. It can be discovered that the elements across all channels for every pixel are independent of other pixels, so we can obtain the solution pixel-by-pixel respectively by appropriate transformation. In terms of the lth $(l = 1, 2, 3, \ldots, L)$ pixel, the optimization equation can be rewritten as follows:

$$\begin{aligned}
\arg\min_{\Theta_l(\hat{\mathbf{h}}_t)} &\frac{1}{2}\left\|\Theta_l(\hat{\mathbf{h}}_t^k)\Theta_l(\hat{\mathbf{s}}_t^k)^\top - (\widehat{\mathbf{m}_t \odot \mathbf{y}})_l\right\|_2^2 + \frac{\eta}{2}\,\|\,\Theta_l(\hat{\mathbf{h}}_t^k) \\
&- \Theta_l(\hat{\mathbf{h}}_{t-1}^k)\,\|_2^2 + \frac{\rho_u}{2}\,\|\,\Theta_j(\hat{\mathbf{h}}_t^k) - \Theta_j(\hat{\mathbf{f}}_{t,u-1}^k) + \Theta_j(\hat{\mathbf{g}}_{t,u-1}^k)\,\|_2^2
\end{aligned}, \tag{15}$$

where $\Theta_l(\cdot) \in \mathbb{C}^{1\times K}$ symbolizes all K values corresponding to the lth pixel. By setting the gradient about $\Theta_l(\hat{\mathbf{h}}_t)$ as zero, Eq. (15) can be derived to:

$$\begin{aligned}
&\Theta_l(\hat{\mathbf{h}}_{t,u}) = \left((\eta + \rho_u)\mathbf{I}_K + \Theta_l(\hat{\mathbf{s}}_t^k)^\top\Theta_l(\hat{\mathbf{s}}_t^k)\right)^{-1} \\
&\left(\rho_u\Theta_l(\hat{\mathbf{f}}_{t,u-1}) + \eta\Theta_l(\hat{\mathbf{h}}_{t-1}) - \rho_u\Theta_l(\hat{\mathbf{g}}_{t,u-1}) + \Theta_l(\hat{\mathbf{s}}_t)(\widehat{\mathbf{m}_t \odot \mathbf{y}})_l\right)
\end{aligned}, \tag{16}$$

where $\mathbf{I}_K \in \mathbb{R}^{K\times K}$ denotes identity matrix. To avoid the inverse matrix operation which brings heavy computation burden, the Sherman–Morrison equation, *i.e.*, $(\mathbf{a}\mathbf{b}^\top + \mathbf{P})^{-1} = \mathbf{P}^{-1} - \frac{\mathbf{P}^{-1}\mathbf{a}\mathbf{b}^\top\mathbf{P}^{-1}}{1+\mathbf{b}^\top\mathbf{P}^{-1}\mathbf{a}}$ is adopted into

the derivation and Eq. (16) can be rewritten as:

$$\Theta_l(\widehat{\mathbf{h}}_{t,u}) = \frac{1}{\eta+\rho_u}\left(\mathbf{I}_K - \frac{\Theta_l(\widehat{\mathbf{s}}_t^k)^\top\Theta_l(\widehat{\mathbf{s}}_t^k)}{\eta+\rho_u+\Theta_l(\widehat{\mathbf{s}}_t^k)\Theta_l(\widehat{\mathbf{s}}_t^k)^\top}\right) \cdot \left(\rho_u\Theta_l(\widehat{\mathbf{f}}_{t,u-1}) + \eta\Theta_l(\widehat{\mathbf{h}}_{t-1}) - \rho_u\Theta_l(\widehat{\mathbf{g}}_{t,u-1}) + \Theta_l(\widehat{\mathbf{s}}_t)(\widehat{\mathbf{m}_t \odot \mathbf{y}})_l\right) \tag{17}$$

By calculating the values across K channels for each pixel of $\widehat{\mathbf{h}}_{t,u}$, this subproblem can be efficiently solved by Eq. (17).

5.2.2. Solving subproblem $\mathbf{f}_t$

Contrasting with the subproblem aforementioned, $\mathbf{f}_{t,u}$ can be obtained easily using $\mathbf{h}_{t,u}$ and $\mathbf{g}_{t,u-1}$. Two blocks of Eq. (12) are adopted for optimization and the objective can be expressed as follows:

$$\arg\min_{\mathbf{f}_t} \frac{1}{2}\sum_{k=1}^{K}\|\mathbf{w}\odot\mathbf{f}_t^k\|_2^2 + \frac{\rho_u}{2}\sum_{k=1}^{K}\|\mathbf{h}_{t,u}^k - \mathbf{f}_t^k + \mathbf{g}_{t,u-1}^k\|_2^2. \tag{18}$$

Considering the independence of each channel in Eq. (18), the filter $\mathbf{f}_{t,u}$ can be solved channel-by-channel. By taking the derivative of every channel in Eq. (18) to be zero, the optimization result can be obtained rapidly in spatial domain which is shown as follows:

$$\begin{aligned}\mathbf{f}_{t,u}^k &= \arg\min_{\mathbf{f}_t^k} \frac{1}{2}\|\mathbf{w}\odot\mathbf{f}_t^k\|_2^2 + \frac{\rho_u}{2}\|\mathbf{h}_{t,u}^k - \mathbf{f}_t^k + \mathbf{g}_{t,u-1}^k\|_2^2 \\ &= \arg\min_{\mathbf{f}_t^k} \frac{1}{2}\|\mathbf{f}_t^k\mathbf{W}\|_2^2 + \frac{\rho_u}{2}\|\mathbf{h}_{t,u}^k - \mathbf{f}_t^k + \mathbf{g}_{t,u-1}^k\|_2^2 \\ &= (\mathbf{W}^\top\mathbf{W} + \rho_u\mathbf{I}_L)^{-1}\rho_u(\mathbf{h}_{t,u}^k + \mathbf{g}_{t,u-1}^k) \\ &= \frac{\rho_u(\mathbf{h}_{t,u}^k + \mathbf{g}_{t,u-1}^k)}{\mathbf{w}\odot\mathbf{w}+\rho_u}\end{aligned}, \tag{19}$$

where we introduce an intermediate variable $\mathbf{W} = diag(\mathbf{w}) \in \mathbb{R}^{L\times L}$ for conciseness and $\mathbf{I}_L \in \mathbb{R}^{L\times L}$ is a identity matrix.

5.2.3. Updating Lagrangian multiplier $\mathbf{g}_t$

After optimizing dual variables $\mathbf{h}_t$ and $\mathbf{f}_t$ in current iteration, the Lagrangian multiplier can be updated using their difference by:

$$\mathbf{g}_{t,u} = \mathbf{g}_{t,u-1} + \rho_u(\mathbf{h}_{t,u} - \mathbf{f}_{t,u}). \tag{20}$$

Remark 2. The penalty factor ρ is preset for each iteration by $\rho_u = min(\rho_{max}, \psi\rho_{u-1})$, where ψ is a step size parameter. In this work, $\rho_1 = 1, \psi = 10$ and $\rho_{max} = 1000$.

Through U iterations of three subproblems mentioned above, the overall problem converges in the end.

5.3. Motion-aware search strategy

In classical methods (Henriques et al., 2015; Danelljan et al., 2015; Danelljan et al., 2004), the new object location can be decided by directly using Eq. (3). However, this kind of detection strategy has two drawbacks. To begin with, the object in the current image possibly exists in the edges of the patch in the fast motion scenarios, and hence easily results in detection failure. Otherwise, since the current object location hardly coincides with the object location in the last image and the boundary effect cannot be eliminated completely, the further the object is from the center point, the more possible the classification result is disturbed by boundary effect. To avoid these conditions to the maximum extent, we propose to introduce the motion information into the search center decision. We define the velocity variable $\mathbf{v}_t$ in tth frame by:

$$\mathbf{v}_t = [\mathbf{v}_{t,x}, \mathbf{v}_{t,y}] = [x_t, y_t] - [x_{t-1}, y_{t-1}] , \tag{21}$$

where $\mathbf{c}_t = [x_t, y_t]$ denotes the predicted location in frame t. Instead of cropping search patch centering at $\mathbf{c}_{t-1}$, this work uses the motion-aware object location $\mathbf{c}_{t,m}$ to be the cropping center with the assumption that the velocity in two consecutive frames does not change, which can be obtained by:

$$\mathbf{c}_{t+1,m} = [\mathbf{v}_{t,x}, \mathbf{v}_{t,y}] + [x_t, y_t] . \tag{22}$$

Remark 3. The search area can be further enlarged properly (25 times larger than the object area in this work) thanks to our proposed distractors repressed method which can effectively alleviate the context interference.

5.4. Scale estimation and algorithm acceleration

For scale adaption, many state-of-the-art DCF trackers (Li et al., 2018; Danelljan et al., 2015; Galoogahi et al., 2017) adopt brute-force multi-scale search strategy in the detection phase which extracted hierarchical feature maps referring to the scale pyramid and decided the optimum scale by comparing the peak value in corresponding response maps. Though easy-operated, such methods take heavy computation on the algorithm for calculating the feature representations on multiply large patches. In this work, we employ the scale filter proposed in Danelljan et al. (2004) to estimate the scale variations. Trained by Eq. (1) and sampling in scale space, the scale filter has a comparable function to multi-scale search strategy while reduces the computational cost to some extent.

By investigating the DCF framework, we discovered that most algorithms respectively extract the feature map for object detection and filter training. Such a strategy, however, doubles the running time of feature extraction per frame and hence is a critical bottleneck for algorithm acceleration. To cope with it, we follow the trick adopted in Danelljan et al. (2017a), in which the feature map utilized for object detection is repeatedly used for filter training after shifting operation, depending on the center offset of the peak. Inspired by this, we additionally propose to extend this idea to the acceleration of scale filter training. The feature representations extracted for scale search are also employed to train scale filter while the scale is unchanged, otherwise, we follow a shifting–cropping–filling strategy. This strategy shifts the sample of current optimum scale to the center according to the current scale variation obtained in the detection phase at first, crops the samples that have not skipped from one end to the other, and extracts new sample to fill the vacancy in the end.

6. Experiments

6.1. Datasets and metrics

We conduct our experiments on three challenging and large-scale UAV tracking datasets publicized recently whose specific details are displayed in Table 1. The sequences in DTB70 dataset (Li and Yeung, 2017) are collected from either videos recorded in a university by drones or Youtube videos, most of which have background clutters or camouflages challenges. The UAV123@10fps dataset (Mueller et al., 2016) is created by downsampling the original 30FPS videos and hence makes it more appropriate to trial the robustness of trackers. Orienting to vehicle tracking from UAV, the videos in UAVDT dataset (Du et al., 2018) provide more challenging factors, *e.g.*, complex weather conditions, multi-level flight altitudes, and various view angles. Therefore, the datasets employed in this work can comprehensively and unbiasedly evaluate the trackers and validate the effectiveness of our methods. For evaluation in each sequence, the bounding boxes of all evaluated tracking methods are initialized with the ground truth in the beginning image, and the per-frame output result is an array containing the object coordinates and size.

For the quantitative contrast of the evaluated trackers, two metrics from one-pass evaluation protocol (Wu et al., 2013) are adopted, *i.e.*, center location error (CLE) and bounding box overlap (BBO). CLE for each frame can be obtained by computing the distance between the predicted location and the ground truth location, while BBO is calculated

Table 1
Details of three benchmarks used for evaluation

Dataset	Sequences	Total frames	Average frames	Conf.
UAV123@10fps (Mueller et al., 2016)	123	37,607	915	ECCV'16
DTB70 (Li and Yeung, 2017)	70	15,777	225	AAAI'17
UAVDT (Du et al., 2018)	50	37,084	742	ECCV'18

Algorithm 1 Overall work-flow of DR^{2}Track

Input: Object location and size in the first image
Successive D images captured by UAV
Output: Object location and size in D frames

```
if t = 1 then
    Sample s_1 centered at the groundtruth location
    Initialize the filter f_1 using Eqs. (17), (19), (20)
    Sample in scale domain and initialize the scale filter
    Set the velocity variable v_t to zero vector.
else
    Calculate the search center location c_{1,m} using Eq. (22)
    Extract the search sample z_t centered at c_{1,m}
    Obtain the response map M_t by Eq. (3)
    Estimate the current object location c_t by finding peak
    Input M_t into the local-maximum-extracted function Ψ(·)
    Obtain the distractor-suppressed vector m_t by Eq. (6)
    Extract the search sample in scale domain
    Estimate the optimum object scale using scale filter
    Shift the z_t to the training sample s_t in spatial domain
    Shift the scale search sample for training in scale domain
    Update the filter f_t using Eq. (17), (19), (20)
    Update the scale filter using the sample obtain by shift
    Calculate the velocity variable v_t using Eq. (21).
end
```

by dividing the union of the output bounding box and the ground truth one by their intersection. In terms of the comparison on entire frames, the precision score (PS) and success rate score (SRS) are respectively calculated for various setting thresholds by counting the numbers of frames whose CLE or BBO are lower than these thresholds. To compare the trackers intuitively, the precision plots (PP) and success rates plots (SRP) are drawn with the x-axis and y-axis denoting the thresholds and scores, respectively. For ranking all tested trackers finally, the normal precision score (NPS) for the threshold of 20-pixel and the area under curve (AUC) of SRP are employed.

6.2. Platform and parameters setting

All of our experiments are processed on MATLAB2018a software installed in a computer possessing an Intel Core i7-8700K CPU with a reference frequency of 3.70 GHz, 32 GB RAM and an NVIDIA Geforce RTX 2080 GPU. Note that the handcrafted feature-based trackers run only with a single CPU and the CNN-based trackers employ GPU for acceleration of convolution and pooling operation.

Following the baseline tracker (Li et al., 2018), 31-channel HOG feature (Dalal and Triggs, 2005), 10-channel CN feature (Weijer and Schmid, 2006) and single-channel gray-scale feature are utilized for appearance representation of the translation estimation filter. For the scale filter, being the same as (Danelljan et al., 2004), this work only employs the HOG feature (Dalal and Triggs, 2005). The cell size is set to four pixels for high-efficient feature extraction. Considering the changeable object scale, the feature maps for translation estimation are resized to 50×50 to keep the temporal dimension-consistency. The shape of the search region is set to be square and its side length is $\theta\sqrt{HW}$ (W and H symbolizes the object width and height, $\theta = 5$ in this work). In terms of temporal restriction, η is set as 18. For ADMM optimization, we set the iteration number U to be 4. β is set as 0.2 for the suppression degree while E is set to 30 to limit the number of repressed local maximums.

Remark 4. The hyper parameters of our DR^{2}Track are preset before the experiments and never changed to keep fair comparison with other trackers. We run the publicized codes of all other evaluated trackers without any modification for fairness.

6.3. Evaluation of handcrafted feature-based trackers

Adopting only handcrafted features, handcrafted feature-based trackers possess an innate advantage over CNN-based ones for their lower computation complexity. In this work, we primarily compare DR^{2}Track with seventeen state-of-the-art handcrafted feature-based methods on CPU and the results are exhibited on Table 2. DR^{2}Track takes the first place on DTB70 and UAV123@10fps datasets (Li and Yeung, 2017; Mueller et al., 2016) and the second place on UAVDT dataset (Du et al., 2018) in both NPS and AUC. On average, the proposed method outperforms all other evaluated trackers without casting aside the real-time attribute. Compared with the first place one (Huang et al., 2019) on UAVDT, DR^{2}Track is only 0.008 and 0.009 lower than it in NPS and AUC, while elevates the average tracking speed by 212%. On DTB70 (Li and Yeung, 2017), DR^{2}Track improves the NPS by 5% and 12.3%, and the AUC by 5.3% and 13.7%, respectively, compared with the second-best one and the baseline tracker (Li et al., 2018). DR^{2}Track also has a notable improvement on UAV123@10fps (Mueller et al., 2016) and UAVDT (Du et al., 2018) compared to the baseline (Li et al., 2018) (9.1%, 13.2% in NPS and 5.5%, 9.2% in AUC). Running over 30FPS (normal frame rate of UAV camera), real-time trackers are the chief choices for UAV tracking. The PP and SRP of twelve evaluated real-times trackers are shown in Fig. 6. It can be spotted that DR^{2}Track achieves the highest PP and SRP on most location error thresholds and overlap thresholds. Compared with the second best real-time trackers (Danelljan et al., 2017a), a mean elevation of 8.4% and 7.7% has been achieved by DR^{2}Track on NPS and AUC.

6.4. Performance comparison with CNN-based trackers

For more comprehensive evaluation of the proposed method, thirteen CNN-based trackers are processed on three benchmarks in this work. The performance contrasts between DR^{2}Track and CNN-based methods are exhibited in Table 3. Neither employing deep features nor applying the end-to-end framework, DR^{2}Track outperforms most CNN-based trackers and achieves the fourth place in both average NPS and AUC. It is noted that in NPS the gap between ours and the best tracker (Bertinetto et al., 2016) is only 0.004. We attribute this remarkable result to our designed adaptive target label, distractors repressed method and motion-aware search strategy. Without the acceleration of high-end GPU, our method still gains the second place in speed, which can be mainly owed to the tighter computation burden of the handcrafted features.

6.5. Attribute-oriented comparison

For further analysis of the performance when DR^{2}Track is against different scenes, we employ the attributes set in the original datasets (Li and Yeung, 2017; Mueller et al., 2016; Du et al., 2018) in our experiments. We divide all attributes of three benchmarks into three parts, i.e., distractor scenarios, fast motion scenarios, and other scenarios and compare the tracking performance of six handcrafted feature-based trackers with ours, respectively.

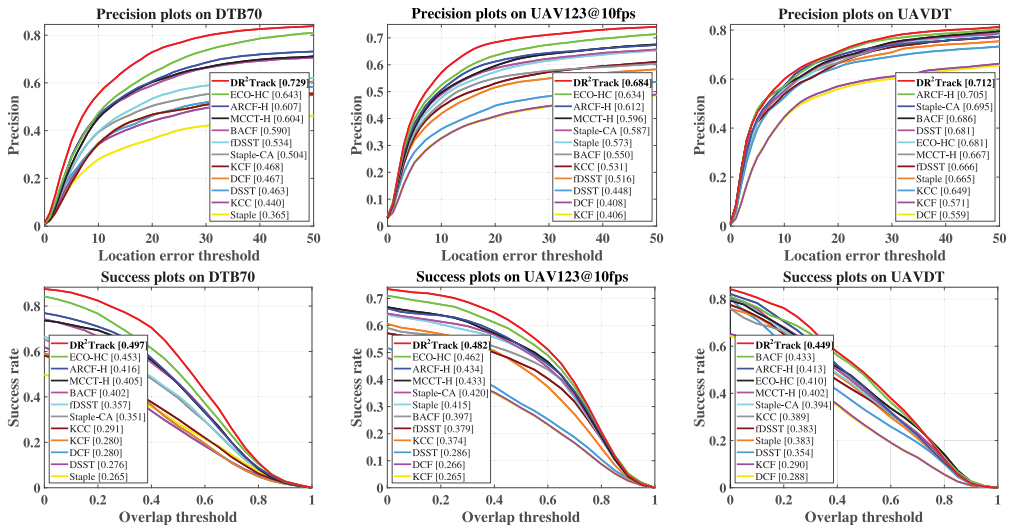

Fig. 6. Overall performance of twelve handcrafted real-time trackers on (a) DTB70 (Li and Yeung, 2017) (b) UAV123@10fps (Mueller et al., 2016) (c) UAVDT (Du et al., 2018). It can be clearly seen that DR^{2}Track ranks first in both precision score and success rate score. Video can be seen on https://youtu.be/49ko4Roexsw, while code for DR^{2}Track can be downloaded on http://github.com/vision4robotics/DR2Track.

Table 2
Comprehensive comparison between 17 handcrafted feature-based trackers and DR^{2}Track on three datasets (Li and Yeung, 2017; Du et al., 2018; Mueller et al., 2016). **Red**, **green**, **blue** and **orange** respectively denote the first, second, third and fourth place. Here, the values in the third column denote the average speed on three datasets. Real-time means that the average speed of trackers is over 30 frames per second. MS and SF respectively denotes two scale estimation methods (multi-scale search and scale filter).

Tracker	Venus	Speed	Real-time	DTB70		UAV123@10fps		UAVDT		MS	SF	Average	
				NPS	AUC	NPS	AUC	NPS	AUC			NPS	AUC
DR^{2}Track	**This work.**	**47.8FPS**	√	**0.729**	**0.497**	**0.684**	**0.482**	**0.712**	**0.449**	×	√	**0.708**	**0.476**
ARCF-H (Huang et al., 2019)	ICCV'19	51.2FPS	√	0.607	0.416	0.612	0.434	**0.705**	0.413	×	√	0.641	0.421
ARCF-HC (Huang et al., 2019)	ICCV'19	15.3FPS	×	**0.694**	**0.472**	**0.666**	**0.473**	**0.720**	**0.458**	×	√	**0.693**	**0.468**
BACF (Galoogahi et al., 2017)	ICCV'17	56.0FPS	√	0.590	0.402	0.550	0.397	**0.686**	**0.433**	√	×	0.609	0.411
CSR-DCF (Lukežic et al., 2017)	CVPR'17	12.1FPS	×	**0.646**	**0.438**	**0.643**	0.450	0.674	0.389	×	√	**0.654**	0.426
DCF (Henriques et al., 2015)	TPAMI'15	**1032.5FPS**	√	0.467	0.280	0.408	0.266	0.559	0.288	×	×	0.526	0.278
DSST (Danelljan et al., 2004)	BMVC'14	**106.5FPS**	√	0.463	0.276	0.448	0.286	0.681	0.354	×	√	0.531	0.305
ECO-HC (Danelljan et al., 2017a)	CVPR'17	69.3FPS	√	0.643	0.453	**0.634**	**0.462**	0.681	0.410	×	√	**0.653**	**0.442**
fDSST (Danelljan et al., 2017b)	TPAMI'17	**168.1FPS**	√	0.534	0.357	0.516	0.379	0.666	0.383	×	√	0.572	0.373
KCC (Wang et al., 2018)	AAAI'18	46.1FPS	√	0.440	0.291	0.531	0.374	0.649	0.389	×	√	0.540	0.351
KCF (Henriques et al., 2015)	TPAMI'15	**615.1FPS**	√	0.468	0.280	0.406	0.265	0.571	0.290	×	×	0.482	0.278
MCCT-H (Wang et al., 2018)	CVPR'18	59.7FPS	√	0.604	0.405	0.596	0.433	0.667	0.402	×	√	0.622	0.413
SAMF (Li and Zhu, 2014)	ECCV'14	12.8FPS	×	0.519	0.340	0.466	0.326	0.579	0.312	√	×	0.521	0.326
SRDCF (Danelljan et al., 2015)	ICCV'15	14.0FPS	×	0.512	0.363	0.575	0.423	0.658	**0.419**	√	×	0.582	0.402
SRDCFdecon (Danelljan et al., 2016)	CVPR'16	7.5FPS	×	0.504	0.351	0.584	0.429	0.643	0.410	√	×	0.577	0.397
Staple (Bertinetto et al., 2016)	CVPR'16	65.4FPS	√	0.365	0.265	0.573	0.415	0.665	0.383	×	√	0.495	0.330
Staple-CA (Mueller et al., 2017)	CVPR'17	58.5FPS	√	0.504	0.351	0.587	0.420	0.504	0.351	×	√	0.595	0.388
STRCF (Li et al., 2018)	CVPR'18	28.5FPS	×	**0.649**	**0.437**	0.627	**0.457**	0.629	0.411	√	×	0.635	**0.435**

Table 3
Comprehensive comparison between 13 recent state-of-the-art CNN-based trackers and DR^{2}Track on three benchmarks (Li and Yeung, 2017; Du et al., 2018; Mueller et al., 2016). **Red**, **green**, **blue** and **orange** respectively denote the first, second, third and fourth place. EE means that this tracker apply end-to-end framework, while DF means deep features are adopted for representation.

Tracker	Venus	DF	EE	DTB70			UAVDT			UAV123@10fps			Average		
				NPS	AUC	FPS	NPS	AUC	FPS	NPS	AUC	FPS	NPS	AUC	FPS
DR^{2}Track	**This work.**	×	×	**0.729**	**0.497**	**47.2**	**0.712**	**0.449**	**49.7**	**0.684**	0.482	**46.4**	**0.708**	**0.476**	**47.8@CPU**
C-COT (Danelljan et al., 2016)	ECCV'16	√	×	**0.769**	**0.517**	1.1	0.656	0.406	1.1	**0.706**	**0.503**	1.1	**0.710**	0.475	1.1@GPU
CFNet (Valmadre et al., 2017)	CVPR'17	×	√	0.571	0.393	**43.5**	0.68	0.428	**41.1**	0.563	0.417	**43.5**	0.605	0.413	**42.7@GPU**
CF2(Ma et al., 2015)	ICCV'15	√	×	0.616	0.415	14.4	0.602	0.355	20.2	0.601	0.425	16.0	0.606	0.398	16.9@GPU
DeepSTRCF (Li et al., 2018)	CVPR'18	√	×	**0.734**	**0.506**	6.2	0.667	0.437	6.6	0.682	**0.499**	6.2	0.694	**0.481**	6.3@GPU
DSiam (Guo et al., 2017)	ICCV'17	×	√	0.727	0.488	8.1	**0.704**	**0.457**	15.9	0.648	0.456	8.1	0.693	0.467	10.7@GPU
ECO (Danelljan et al., 2017a)	CVPR'17	√	×	0.722	0.502	11.3	**0.700**	**0.454**	16.4	**0.711**	**0.520**	12.7	**0.711**	**0.492**	13.5@GPU
IBCCF (Li et al., 2017)	ICCVW'17	√	×	0.669	0.460	2.2	0.603	0.388	3.4	0.651	0.481	2.4	0.641	0.443	2.7@GPU
MCPF (Zhang et al., 2017)	CVPR'17	√	×	0.664	0.433	0.6	0.660	0.403	0.7	0.665	0.445	0.6	0.663	0.427	0.6@GPU
MCCT (Wang et al., 2018)	CVPR'18	√	×	0.725	0.484	8.6	0.671	0.437	8.6	0.684	0.492	8.4	0.693	0.471	8.5@GPU
SiameseFC (Bertinetto et al., 2016)	ECCV'16	×	√	**0.747**	**0.508**	39.1	**0.708**	**0.465**	37.9	0.68	0.472	39.0	**0.712**	**0.482**	38.7@GPU
TADT (Li et al., 2019)	CVPR'19	×	√	0.693	0.464	35.3	0.677	0.431	32.5	**0.687**	**0.508**	34.0	0.686	0.468	33.9@GPU
UDT (Wang et al., 2019)	CVPR'19	×	√	0.602	0.422	**57.5**	0.674	0.441	**76.4**	0.575	0.430	**61.0**	0.617	0.435	**65.0@GPU**
UDT+(Wang et al., 2019)	CVPR'19	×	√	0.658	0.462	**43.4**	0.674	0.416	**60.4**	0.675	0.478	**39.1**	0.669	0.452	**47.6@GPU**

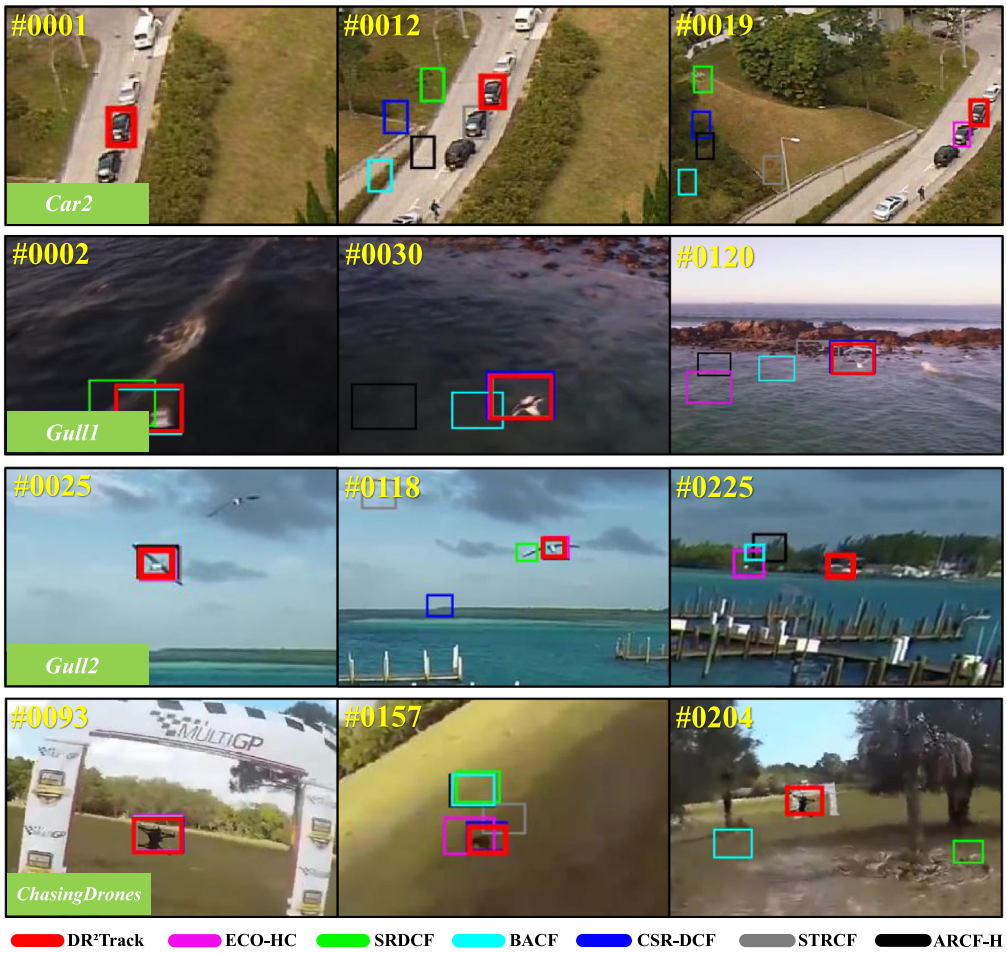

Fig. 7. Qualitative performance of seven handcrafted feature-based trackers on four sequences where distractors exist, *i.e.*, *Car2, Gull1, Gull2, ChasingDrones* from Li and Yeung (2017).

6.5.1. Comparison in distractor scenarios

Among all the attributes in three benchmarks, four attributes are separated into distractor scenarios in this work. The SRP of the evaluated trackers can be seen as exhibited in Fig. 8 and some visual comparisons of tracking results are shown in Fig. 7. In scenes where similar objects exist, the distraction is generated from objects in the same category, while in background clutter, the complexity of context leads to numerous distractors. With distractors monitoring and suppression strategy, DR^{2}Track can well cope with all these scenarios. For example, as shown in Fig. 7, in *Car2* sequence, the output bounding box of other trackers drifts to either other black cars or surrounding plants, however, DR^{2}Track shows strong robustness and tracks the target object accurately. In the *Gull2* sequence, numbers of clutters existing in background, such as cloud, woods and beach, which greatly weaken the distinguishing ability of other trackers, but cannot influence the stable tracking of DR^{2}Track. Notably, in terms of the similar object around attribute of DTB70 (Li and Yeung, 2017), DR^{2}Track elevates the AUC of the second place one (Li et al., 2018) by 13.9% which vigorously demonstrates its robustness against distractors.

6.5.2. Comparison in fast motion scenarios

UAV tracking suffers severely from fast camera or object motion due to higher relative motion DoF and boarder scope. We categorize four attributes to fast motion scenarios and draw the SRP of the evaluated trackers as shown in Fig. 10. DR^{2}Track respectively achieves an improvement of AUC by 9.9%, 2.9%, 17.9% and 14.2%, compared with the baseline tracker (Li et al., 2018). Some qualitative comparison of tracking results can be seen in Fig. 9. Owing to the motion-aware search center decision method, even in extreme conditions of fast motion, DR^{2}Track can still keep a normal tracking, and we can take the *uav3* sequence for instance. The tracked UAV voyages for a long distance only within forty frames, as a result, all trackers undergo failure one-by-one other than DR^{2}Track.

6.5.3. Comparison in other scenarios

We also select five other attributes in each dataset for comparison of NPS and AUC, as shown in Fig. 11. DR^{2}Track takes the first place in thirteen attributes in terms of NPS, while ranks first on AUC in all selected attributes. This superiority of DR^{2}Track over other trackers can be attributed to the effectiveness of our proposed adaptive target label and motion-aware search strategy for the enhancement of tracking robustness produced by them.

6.6. Components effectiveness analysis

6.6.1. Analysis of distractor repression

Compared with the STRCF tracker (Li et al., 2018), we validate the distractor repression method in this work on three datasets. For a fair comparison, we remove the motion-aware search strategy and only retain the local maximum repression (LMR) strategy in DR^{2}Track, which devises a novel DR^{2}Track_LMR method. As exhibited in Table 4, DR^{2}Track gains an improvement of 8.2% in NPS and 6.4% in AUC compared with STRCF. We owe this marked elevation to the repression effect of local maximum values in the response map. Fig. 12 visualizes the variances of the average local maximum peak ratio (ALMPR) of response maps generated by STRCF and DR^{2}Track_LMR

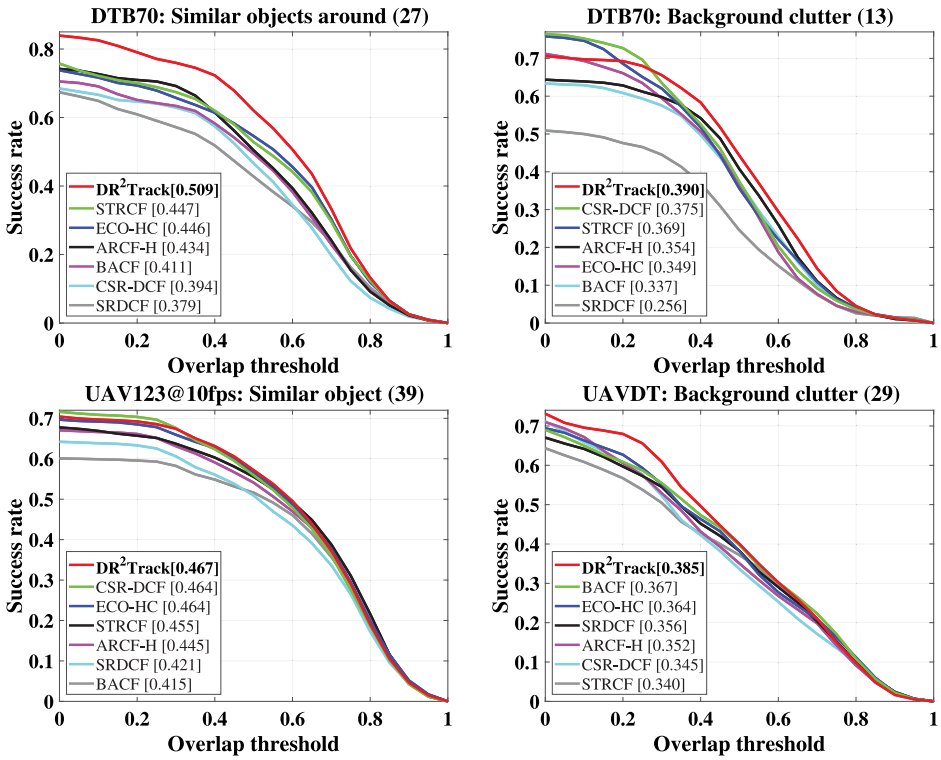

Fig. 8. Success rate plots of seven handcrafted feature-based trackers in four distractors scenario attributes, *i.e.*, background clutter and similar objects around attribute in DTB70 (Li and Yeung, 2017), similar object attribute in UAV123@10fps (Mueller et al., 2016) and background clutter attribute in UAVDT (Du et al., 2018).

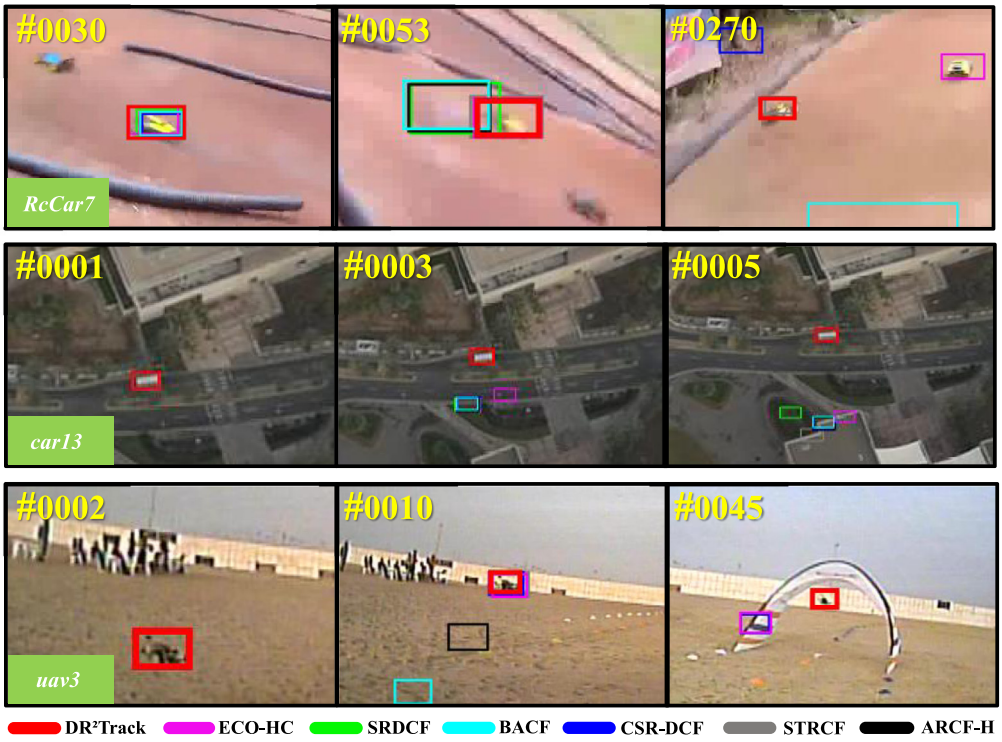

Fig. 9. Qualitative tracking performance of DR²Track and six handcrafted feature-based trackers on three sequences with fast motion, *i.e.*, *RcCar7* from DTB70 (Li and Yeung, 2017) and *car13, uav3* from UAV123@10fps.

in six sequences. It can be clearly seen that when the similar objects get close to the target object in sequences like *group1_1, S0302*, or the clutters and camouflages increase, like the scenarios in *ChasingDrones, Gull2, RcCar5*, the ALMPR of STRCF usually tends to fluctuate around a higher value than that of DR²Track. This phenomenon aggressively and indubitably demonstrates the distractor-repressed effects brought by on-the-fly monitoring and suppression of local maximums and the stronger robustness against capricious scenes therefrom.

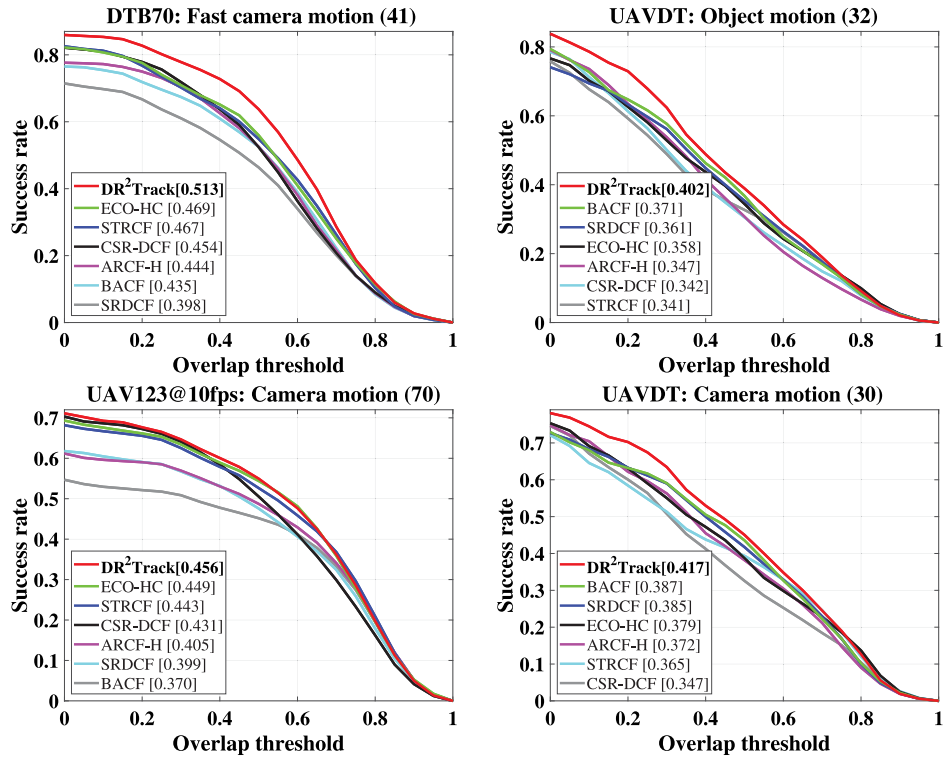

Fig. 10. Success rate plots of seven evaluated trackers in fast camera motion of DTB70 (Li and Yeung, 2017), camera motion of UAV123@10fps (Mueller et al., 2016) and object motion, camera motion from UAVDT (Du et al., 2018). DR²Track ranks first in all of these attributes while our baseline (Li et al., 2018), only ranks the third, the sixth or the last place.

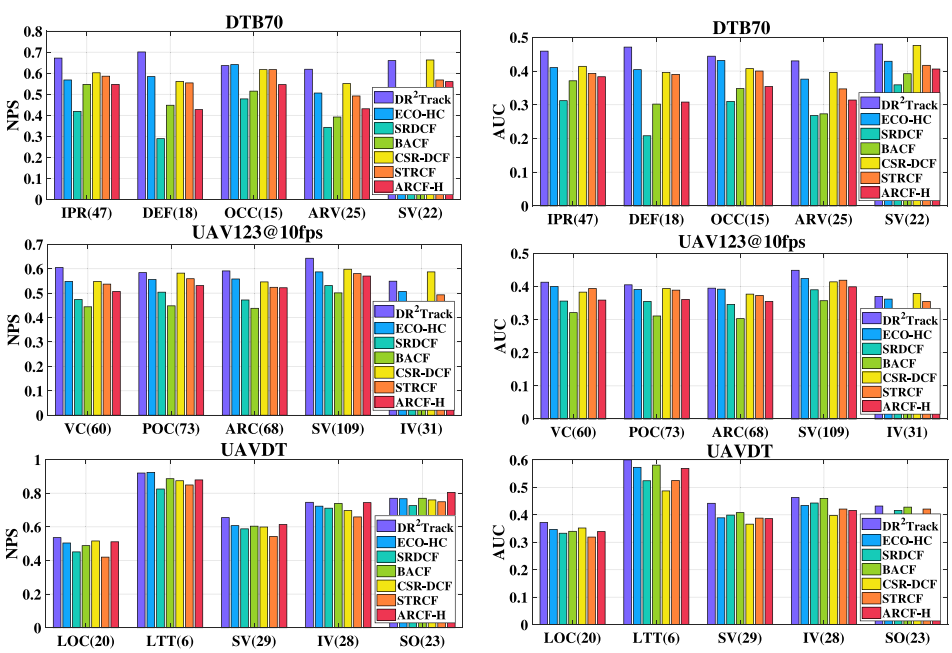

Fig. 11. Comparison of NPS and AUC of seven handcrafted feature-based trackers in fifteen attributes, *i.e.*, in-plane rotation (IPR), deformation (DEF), occlusion (OCC), aspect ratio variation (ARV) and scale variation (SV) from DTB70 (Li and Yeung, 2017), view-point change (VC), partial occlusion (POC), aspect ratio change (ARC), scale variation (SV) and illumination variation (IV) from UAV123@10fps (Mueller et al., 2016), and large occlusion (LOC), long-term tracking (LTT), scale variation (SV), illumination variation (IV) and small occlusion (SO) from UAVDT (Du et al., 2018). Noted that the number behind the attribute name denotes the number of sequences.

6.6.2. Analysis of motion-aware strategy

Also, we analyze the proposed motion-aware search center decision strategy by applying it to the baseline tracker (Li et al., 2018). The comparison results are shown in Table 5. With this component, the NPS and AUC respectively have an average improvement of 2.5% and 1.8% on three datasets, which proves its effectiveness in elevating tracking performance by addressing fast motion problems.

6.6.3. Parameters analysis

In this work, the values of both the number of extracted local maximums E and the repression degree factor β have an influence

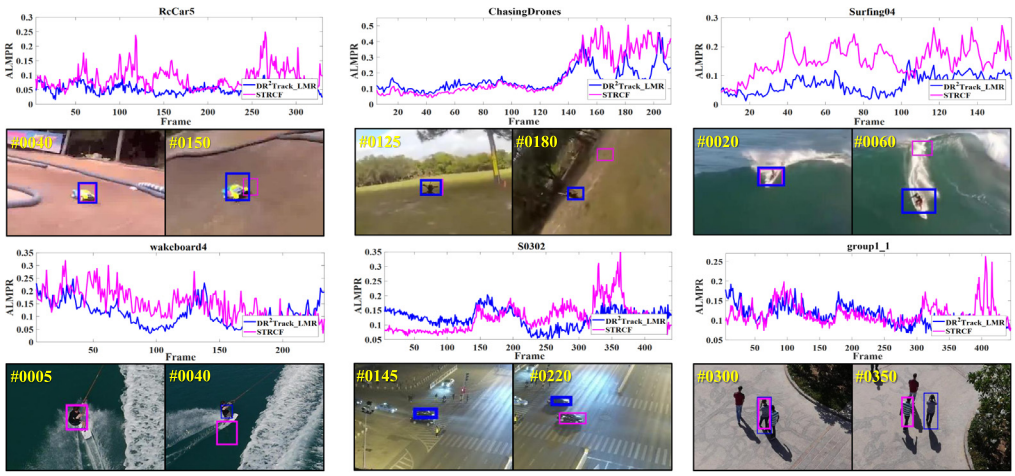

Fig. 12. Sequential comparison of average local maximum peak ratio (ALMPR) between DR^{2}Track_LMR and STRCF on six sequences, *i.e.*, *ChasingDrones,Gull2, RcCar5* from DTB70 (Li and Yeung, 2017), *wakeboard4,group1_1* from UAV123@10fps (Mueller et al., 2016), *S0302* from UAVDT (Du et al., 2018).

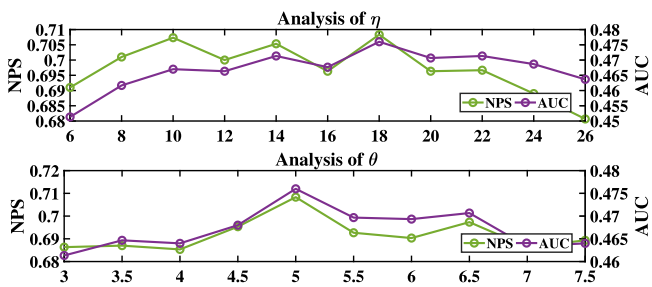

Fig. 13. The impacts on mean tracking performance on three benchmarks (Li and Yeung, 2017; Mueller et al., 2016; Du et al., 2018) of various values of η and θ.

Table 4
Average and individual comparison between DR^{2}Track_LMR with STRCF on NPS and AUC in three benchmarks.

Dataset	NPS		AUC	
	STRCF	DR^{2}Track_LMR	STRCF	DR^{2}Track_LMR
UAV123@10fps (Mueller et al., 2016)	0.627	0.663	0.457	0.475
DTB70 (Li and Yeung, 2017)	0.649	0.700	0.437	0.480
UAVDT (Du et al., 2018)	0.629	0.697	0.411	0.435
Average	0.635	0.687	0.435	0.463

Table 5
Average and individual comparison between STRCF and STRCF+MA on NPS and AUC in three benchmarks.

Dataset	NPS		AUC	
	STRCF	STRCF+MA	STRCF	STRCF+MA
DTB70 (Mueller et al., 2016)	0.646	0.673	0.437	0.451
UAVDT (Li and Yeung, 2017)	0.629	0.656	0.411	0.425
UAV123@10fps (Du et al., 2018)	0.627	0.624	0.457	0.452
Average	0.635	0.651	0.435	0.443

on the distractors repression, and then affect the tracking results. We compare the average tracking performance of DR^{2}Track with various values in E and β, as shown in Table 6. Ranging from 0.703 to 0.708 and 0.474 to 0.476, NPS and AUC are weakly influenced by the value of E, when E is bigger than 10. The reason for this is that only the top few local maximums have a prominent distraction, and hence the repression of other lower local maximums seldom has an influence on the output. Also, the value of β has little impact in the tracking

Table 6
Average tracking performance in three benchmarks (Li and Yeung, 2017; Mueller et al., 2016; Du et al., 2018) when E and β have various values.

E	10	20	**30**	40	50	60
NPS	0.703	0.704	**0.708**	0.707	0.708	0.707
AUC	0.474	0.475	**0.476**	0.475	0.475	0.475
β	0.1	0.15	**0.2**	0.25	0.3	0.35
NPS	0.706	0.706	**0.708**	0.702	0.702	0.700
AUC	0.472	0.474	**0.476**	0.471	0.472	0.470

Table 7
Tracking speed comparison with various acceleration enabled of DR^{2}Track and the baseline tracker STRCF (Li et al., 2018).

Dataset	DR^{2}Track	DR^{2}Track_TS	DR^{2}Track_SS	STRCF
DTB70	47.2	42.2	38.4	26.3
UAV123@10fps	46.4	46.3	37.6	26.9
UAVDT	49.7	49.2	40.5	32.3
Mean	47.8	45.9	38.8	28.4

performance with AUC from 0.470 to 0.476 and NPS from 0.700 to 0.708, which shows the strong robustness of our tracker against the changes of parameters. In Fig. 13, we exhibit the analysis of the values of the temporal regularization parameter η and the search region size factor θ. With the value of η from 8 to 22, the value of NPS and AUC has a relatively small fluctuation between 0.696 to 0.708 and 0.461 to 0.476, respectively. However, due to under or over-restriction of the filter, the performance drops swiftly when η is 6 or 26. As for θ, lower value results in worse performance with the reduction of negative samples, while higher value has smaller impacts.

6.6.4. Acceleration methods analysis

In this work, we apply two methods for algorithm acceleration, *i.e.*, spatial sample reutilization (Danelljan et al., 2017a) and size sample reuse by proposed shifting–cropping–filling strategy. With only shifting spatial sample for translation filter training, we devise a DR^{2}Track_TS method, while only reusing the size sample, we develop a novel DR^{2}Track_SS method. In Table 7, we compare the tracking speed on three datasets (Li and Yeung, 2017; Mueller et al., 2016; Du et al., 2018) when different acceleration modules are enabled. It can be noticed that the proposed shifting–cropping–filling strategy provides a improvement of 1.9FPS on tracking speed. The speed of STRCF (Li et al., 2018) cannot meet the real-time requirements due to its high-cost multi-scale search strategy and extracting samples repeatedly.

7. Conclusions

This work proposes to monitor the local maximums in the response map generated in the detection phase based on the observation that these local maximums have relatively high values and sometimes result in model drift by overturning the peak value. In the training phase, we adaptively revise the fixed regression target by abating the desired values in corresponding locations in order to weaken the distraction brought by stuff having a similar appearance to the object. Consequently, our tracker has stronger robustness against volatile scenarios in UAV object tracking. In consideration of the frequently fast motion in UAV tracking, we present a simple but useful search area decision strategy to tackle the abrupt tremble of UAV or acceleration of the object. Specifically, we design a shifting–cropping–filling scale sample reuse method to elevate the tracking speed without losing accuracy. We conduct experiments on three benchmarks comprehensively and vigorously to prove the effectiveness of our innovations. To our excitement, the performance of DR^{2}Track compares favorably with state-of-the-art trackers using handcrafted features, even superior to some CNN-based trackers. With its procession only on an affordable CPU, DR^{2}Track shows a real-time speed of near 50FPS. We can undoubtedly conclude that our proposed tracker is an excellent alternative for real-world UAV tracking applications and believe that our work will promote the development of real-time image processing.

CRediT authorship contribution statement

Changhong Fu: Conceptualization, Formal analysis, Funding acquisition, Methodology, Project administration, Supervision, Validation, Writing - review & editing. **Fangqiang Ding:** Conceptualization, Formal analysis, Investigation, Methodology, Writing - original draft. **Yiming Li:** Formal analysis, Methodology, Writing - original draft. **Jin Jin:** Data curation, Methodology, Writing - original draft. **Chen Feng:** Methodology, Supervision, Validation, Writing - review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The work was supported by the State Key Laboratory of Mechanical Transmissions, China (Chongqing University) under Grant SKLMT-KFKT-201802 and the National Natural Science Foundation of China under Grant 61806148.

References

Ahmad, A., Lawless, G., Lima, P., 2017. An online scalable approach to unified multirobot cooperative localization and object tracking. IEEE Trans. Robot. 33 (5), 1184–1199.

Babenko, B., Yang, M., Belongie, S., 2009. Visual tracking with online multiple instance learning. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 983–990.

Bertinetto, L., Valmadre, J., Golodetz, S., Miksik, O., Torr, P.H.S., 2016. Staple: Complementary learners for real-time tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 1401–1409.

Bertinetto, L., Valmadre, J., Henriques, J.F., Vedaldi, A., Torr, P.H., 2016. Fully-convolutional siamese networks for object tracking. In: Proceedings of European Conference on Computer Vision, pp. 850–865.

Bharati, S., Wu, Y., Sui, Y., Padgett, C., Wang, G., 2018. Real-time obstacle detection and tracking for sense-and-avoid mechanism in UAVs. IEEE Trans. Intell. Veh. 3 (2), 185–197.

Bolme, D.S., Beveridge, J.R., Draper, B.A., Lui, Y.M., 2010. Visual object tracking using adaptive correlation filters. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 2544–2550.

Comaniciu, D., Ramesh, V., Meer, P., 2000. Real-time tracking of non-rigid objects using mean shift. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 142–149.

Dai, K., Wang, D., Lu, H., Sun, C., Li, J., 2019. Visual tracking via spatially-regularized correlation filters. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4665–4674.

Dalal, N., Triggs, B., 2005. Histograms of oriented gradients for human detection. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, vol. 1, pp. 886–893.

Danelljan, M., Bhat, G., Khan, F.S., Felsberg, M., 2017a. ECO: Efficient convolution operators for tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 6931–6939.

Danelljan, M., Häger, G., Khan, F.S., Felsberg, M., 2004. Accurate scale estimation for robust visual tracking. In: Proceedings of British Machine Vision Conference.

Danelljan, M., Häger, G., Khan, F.S., Felsberg, M., 2015. Learning spatially regularized correlation filters for visual tracking. In: Proceedings of IEEE International Conference on Computer Vision, pp. 4310–4318.

Danelljan, M., Häger, G., Khan, F.S., Felsberg, M., 2016. Adaptive decontamination of the training set: A unified formulation for discriminative visual tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 1430–1438.

Danelljan, M., Häger, G., Khan, F.S., Felsberg, M., 2017b. Discriminative scale space tracking. IEEE Trans. Pattern Anal. Mach. Intell. 39 (8), 1561–1575.

Danelljan, M., Khan, F.S., Felsberg, M., van de Weijer, J., 2014. Color attributes for real-time visual tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 1090–1097.

Danelljan, M., Robinson, A., Khan, F.S., Felsberg, M., 2016. Beyond correlation filters: Learning continuous convolution operators for visual tracking. In: Proceedings of European Conference on Computer Vision, pp. 472–488.

Du, D., Qi, Y., Yu, H., Yang, Y., Duan, K., Li, G., Zhang, W., Huang, Q., Tian, Q., 2018. The unmanned aerial vehicle benchmark: Object detection and tracking. In: Proceedings of European Conference on Computer Vision, pp. 370–386.

Fu, C., Huang, Z., Li, Y., Duan, R., Lu, P., 2019. Boundary effect-aware visual tracking for UAV with online enhanced background learning and multi-frame consensus verification. In: Proceedings of IEEE/RSJ International Conference on Intelligent Robots and Systems, pp. 4415–4422.

Fu, C., Li, B., Ding, F., Lin, F., Lu, G., 2020. Correlation filter for UAV-based aerial tracking: A review and experimental evaluation. ArXiv preprint arXiv:2010.06255.

Fu, C., Zhang, Y., Duan, R., Xie, Z., 2018. Robust scalable part-based visual tracking for UAV with background-aware correlation filter. In: Proceedings of IEEE International Conference on Robotics and Biomimetics, pp. 2245–2252.

Galoogahi, H., Fagg, A., Lucey, S., 2017. Learning background-aware correlation filters for visual tracking. In: Proceedings of IEEE International Conference on Computer Vision, pp. 1144–1152.

Gao, P., Zhang, Q., Wang, F., Xiao, L., Fujita, H., Zhang, Y., 2020. Learning reinforced attentional representation for end-to-end visual tracking. Inform. Sci. 517, 52–67.

Grabner, H., Leistner, C., Bischof, H., 2008. Semi-supervised on-line boosting for robust tracking. In: Proceedings of European Conference on Computer Vision, pp. 234–247.

Guo, Q., Feng, W., Zhou, C., Huang, R., Wan, L., Wang, S., 2017. Learning dynamic siamese network for visual object tracking. In: Proceedings of IEEE International Conference on Computer Vision, pp. 1781–1789.

Hare, S., Saffari, A., Torr, P.H., 2011. Stuck: Structured output tracking with kernels. In: Proceedings of IEEE International Conference on Computer Vision, pp. 263–270.

Held, D., Thrun, S., Savarese, S., 2016. Learning to track at 100 fps with deep regression networks. In: Proceedings of European Conference on Computer Vision, pp. 749–765.

Henriques, J.F., Caseiro, R., Martins, P., Batista, J., 2015. High-speed tracking with kernelized correlation filters. IEEE Trans. Pattern Anal. Mach. Intell.

Huang, Z., Fu, C., Li, Y., Lin, F., Lu, P., 2019. Learning aberrance repressed correlation filters for real-time uav tracking. In: Proceedings of IEEE International Conference on Computer Vision, pp. 2891–2900.

Karakostas, I., Mademlis, I., Nikolaidis, N., Pitas, I., 2019. Shot type constraints in UAV cinematography for autonomous target tracking. Inform. Sci. 506, 273–294.

Khan, S., Xu, G., Chan, R., Yan, H., 2017. An online spatio-temporal tensor learning model for visual tracking and its applications to facial expression recognition. Expert Syst. Appl. 90, 427–438.

Kuai, Y., Wen, G., Li, D., 2018. When correlation filters meet fully-convolutional Siamese networks for distractor-aware tracking. Signal Process. Image Commun. 64, 107–117.

Kuai, Y., Wen, G., Li, D., 2019. Masked and dynamic siamese network for robust visual tracking. Inform. Sci. 503, 169–182.

Li, M., Chen, W., Huang, K., Tan, T., 2010. Visual tracking via incremental self-tunning particle filtering on the affine group. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 1315–1322.

Li, Y., Fu, C., Ding, F., Huang, Z., Lu, G., 2020a. AutoTrack: Towards high-performance visual tracking for UAV with automatic spatio-temporal regularization. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 11923–11932.

Li, Y., Fu, C., Huang, Z., Zhang, Y., Pan, J., 2020b. Intermittent contextual learning for keyfilter-aware UAV object tracking using deep convolutional feature. IEEE Trans. Multmedia 1–13.

Li, X., Ma, C., Wu, B., He, Z., Yang, M., 2019. Target-aware deep tracking. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 1369–1378.

Li, F., Tian, C., Zuo, W., Zhang, L., Yang, M., 2018. Learning spatial-temporal regularized correlation filters for visual tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 4904–4913.

Li, F., Yao, Y., Li, P., Zhang, D., Zuo, W., Yang, M., 2017. Integrating boundary and center correlation filters for visual tracking with aspect ratio variation. In: Proceedings of IEEE International Conference on Computer Vision Workshop, pp. 2001–2009.

Li, S., Yeung, D.-Y., 2017. Visual object tracking for unmanned aerial vehicles: A and new motion models. In: Proceedings of AAAI Conference on Artifical Intelligence, pp. 4140–4146.

Li, Y., Zhu, J., 2014. A scale adaptive kernel correlation filter tracker with feature integration. In: Proceedings of European Conference on Computer Vision, pp. 254–265.

Lukežic, A., Vojír, T., Zajc, L.C., Matas, J., Kristan, M., 2017. Discriminative correlation filter with channel and spatial reliability. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4847–4856.

Ma, C., Huang, J., Yang, X., Yang, M., 2015. Hierarchical convolutional features for visual tracking. In: Proceedings of IEEE International Conference on Computer Vision, pp. 3074–3082.

Mei, X., Ling, H., 2009. Robust visual tracking using L1 minimization. In: Proceedings of IEEE International Conference on Computer Vision, pp. 1436–1443.

Mueller, M., Smith, N., Ghanem, B., 2016. A benchmark and simulator for uav tracking. In: Proceedings of European Conference on Computer Vision, pp. 445–461.

Mueller, M., Smith, N., Ghanem, B., 2017. Context-aware correlation filter tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 1387–1395.

Pantrigo, J., Hernández, J., Sánchez, A., 2010. Multiple and variable target visual tracking for video-surveillance applications. Pattern Recognit. Lett. 31 (12), 1577–1590.

Qi, Y., Zhang, S., Qin, L., Yao, H., Huang, Q., Lim, J., Yang, M., 2016. Hedged deep tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 4303–4311.

Ross, Lim, J., Lin, R., 2004. Incremental learning for visual tracking. In: Proceedings of Advances in Neural Information Processing Systems, pp. 793–800.

Valmadre, J., Bertinetto, L., Henriques, J., Vedaldi, A., Torr, P.H., 2017. End-to-end representation learning for correlation Filter based tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 2805–2813.

Wang, N., Song, Y., Ma, C., Zhou, W., Liu, W., Li, H., 2019. Unsupervised deep tracking. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 1308–1317.

Wang, C., Zhang, L., Xie, L., Yuan, J., 2018. Kernel cross-correlator. In: Proceedings of AAAI Conference on Artificial Intelligence, pp. 1–8.

Wang, N., Zhou, W., Tian, Q., Hong, R., Wang, M., Li, H., 2018. Multi-cue correlation filters for robust visual tracking. In: Proceedings of IEEE/CVF Conference on Computer Vision and Pattern Recognition, pp. 4844–4853.

Weijer, J.V.D., Schmid, C., 2006. Coloring local feature extraction. in: Proceedings of European Conference on Computer Vision, pp. 334–348.

Wu, Y., Lim, J., Yang, M., 2013. Online object tracking: A benchmark. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 2411–2418.

Xiao, J., Qiao, L., Stolkin, R., Leonaridis, A., 2016. Distractor-supported single target tracking in extremely cluttered scenes. In: Proceedings of European Conference on Computer Vision, pp. 121–136.

Zhang, T., Xu, C., Yang, M.-H., 2017. Multi-task correlation particle filter for robust object tracking. In: Proceedings of IEEE Conference on Computer Vision and Pattern Recognition, pp. 4335–4343.

Zhao, D., Xiao, L., Fu, H., Wu, T., Xu, X., Dai, B., 2019. Augmenting cascaded correlation filters with spatial–temporal saliency for visual tracking. Inform. Sci. 470, 78–93.